Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

MCS 481 Lecture 29
Computational Geometry
Jan Verschelde, 30 October 2026

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

definition and complexity

Definition (the Delaunay triangulation)

Given a point set P, *the Delaunay triangulation of P* maximizes the minimal angle over all triangulations of P.

The complexity is linear in the size of the input.

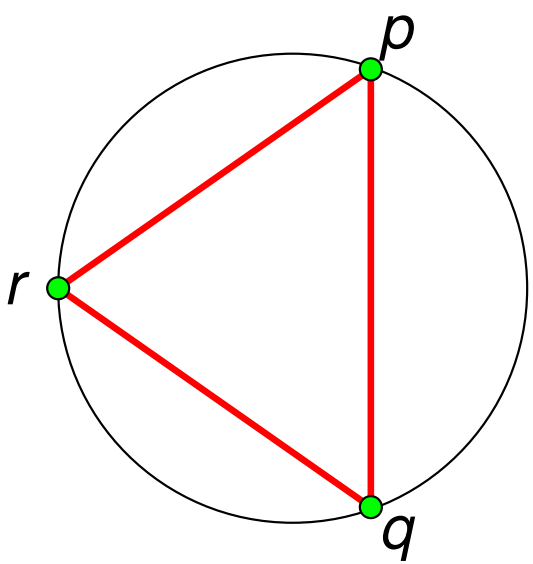
Theorem (complexity of a triangulation of a point set)

Let P be a set of n points, not all on the same line.
Let k be the number of points on the boundary of the convex hull of P.
Any triangulation of P has

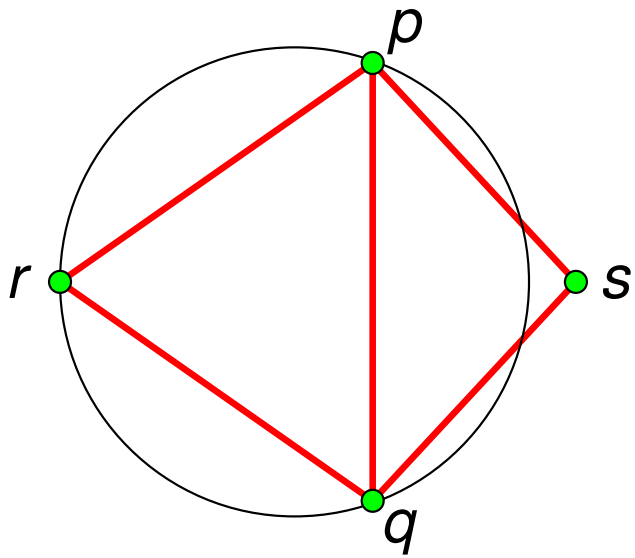
1. $2n - 2 - k$ *triangles, and*
2. $3n - 3 - k$ *edges.*

circumcircle characterization

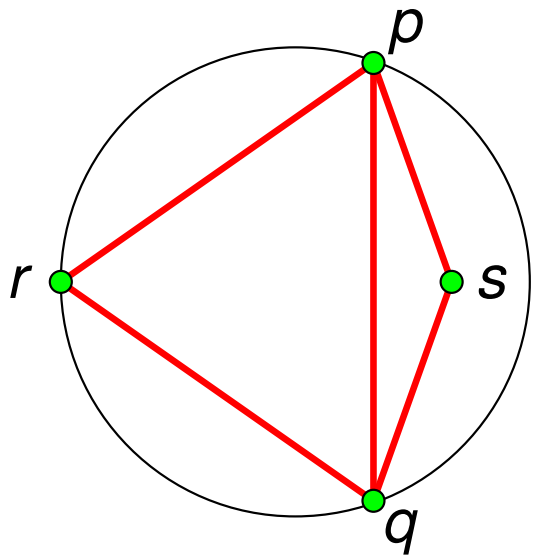
The three vertices of a triangle define a circle uniquely.



circumcircle



legal



illegal

Theorem (circumcircle characterization)

A triangulation T of P is Delaunay $\Leftrightarrow$ the circumcircle of any triangle of T does not contain a point of P in its interior.

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

affine dependencies and edge flips

Consider four points p, q, r, and s, not all on the same line, with coordinates $p = (p_x, p_y)$, $q = (q_x, q_y)$, $r = (r_x, r_y)$, $s = (s_x, s_y)$.

$$\begin{array}{ccccccccc} \begin{pmatrix} 0 \\ 0 \end{pmatrix} & = & \lambda_p \begin{pmatrix} p_x \\ p_y \end{pmatrix} & + & \lambda_q \begin{pmatrix} q_x \\ q_y \end{pmatrix} & + & \lambda_r \begin{pmatrix} r_x \\ r_y \end{pmatrix} & + & \lambda_s \begin{pmatrix} s_x \\ s_y \end{pmatrix} \\ 0 & = & \lambda_p & + & \lambda_q & + & \lambda_r & + & \lambda_s \end{array}$$

is an affine dependency, with $(\lambda_p, \lambda_q, \lambda_r, \lambda_s) \neq (0, 0, 0, 0)$, unique up to a nonzero multiple. There are two triangulations: T_+ and T_-:

1. T_+ consists of all triangles with one point removed for which $\lambda > 0$.
2. T_- consists of all triangles with one point removed for which $\lambda < 0$.

two configurations of four points

Exercise 1: For $p = (0,0)$, $q = (1,0)$, $r = (0,1)$, $s = (1,1)$, compute $(\lambda_p, \lambda_q, \lambda_r, \lambda_s)$.

1. Draw the two triangulations T_+ and T_- defined by the signs of $(\lambda_p, \lambda_q, \lambda_r, \lambda_s)$.
2. Describe the relationship between T^+ and T^- and the edge flips.

Exercise 2: For $p = (0,0)$, $q = (2,0)$, $r = (1,1)$, $s = (1,2)$, compute $(\lambda_p, \lambda_q, \lambda_r, \lambda_s)$.

1. Draw the two triangulations T_+ and T_- defined by the signs of $(\lambda_p, \lambda_q, \lambda_r, \lambda_s)$.
2. Describe the relationship between T^+ and T^- and the edge flips.

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

a randomized incremental algorithm

Algorithm DELAUNAYTRIANGULATION(P)

Input: P, a set of $n+1$ points, $P = \{ p_0, p_1, \ldots, p_n \}$.
Output: T, a Delaunay triangulation of P.

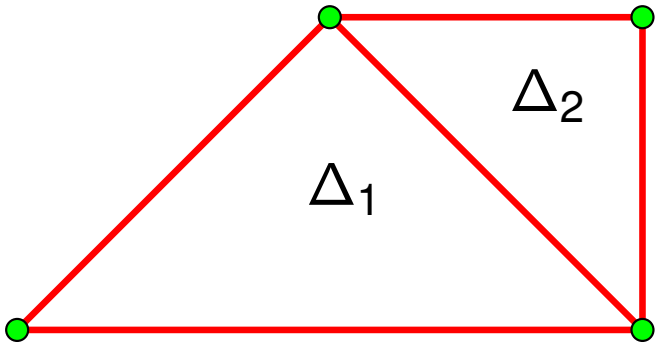
1. let p_0 be the rightmost highest point in P
2. let p_{-1} and p_{-2} be such that $P \subseteq \Delta p_{-2} p_{-1} p_0$
3. $T = \{ \Delta p_{-2} p_{-1} p_0 \}$
4. compute a random permutation of $P \setminus \{ p_0 \}$
5. for r from 1 to n do
6. locate $\Delta p_i p_j p_k \in T$: $p_r \in \Delta p_i p_j p_k$
7. UPDATE($T, p_r, \Delta p_i p_j p_k$)

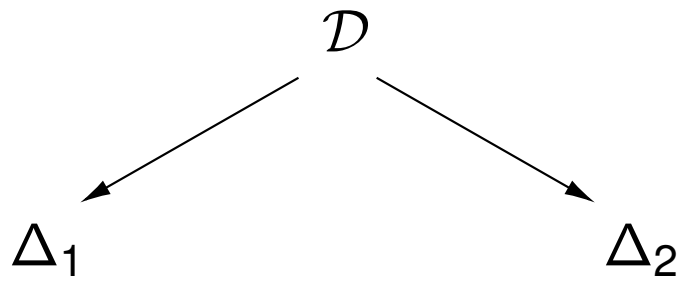
Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

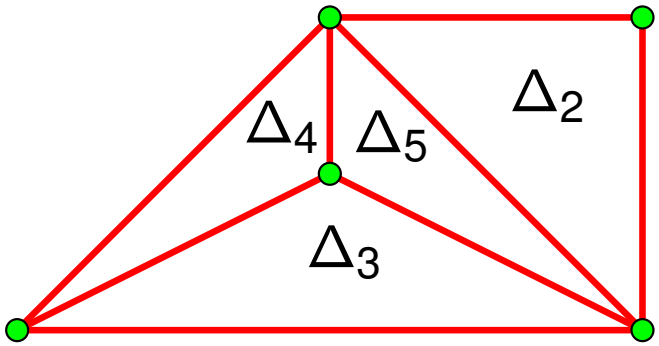
the directed acyclic graph for point location

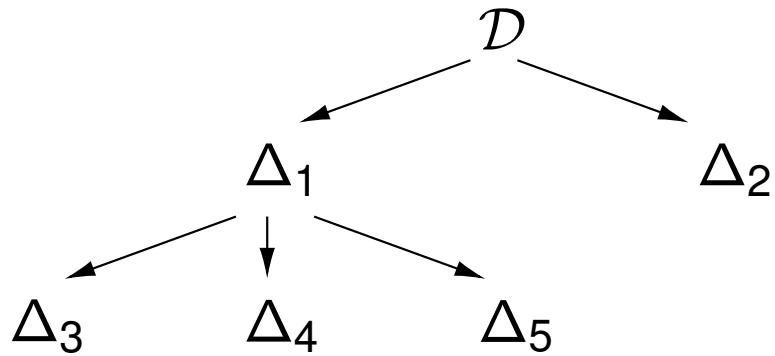
Edges between triangles determine the navigation in the graph.
Every triangle is stored only once.



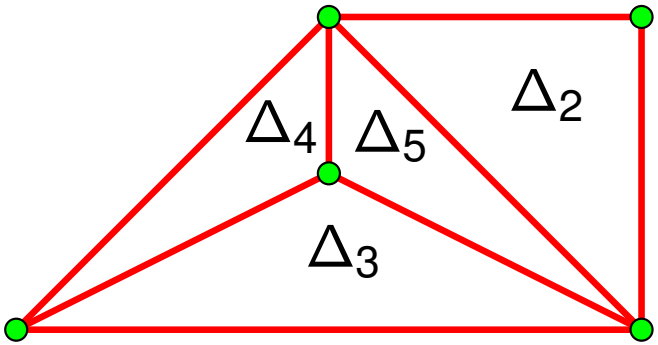


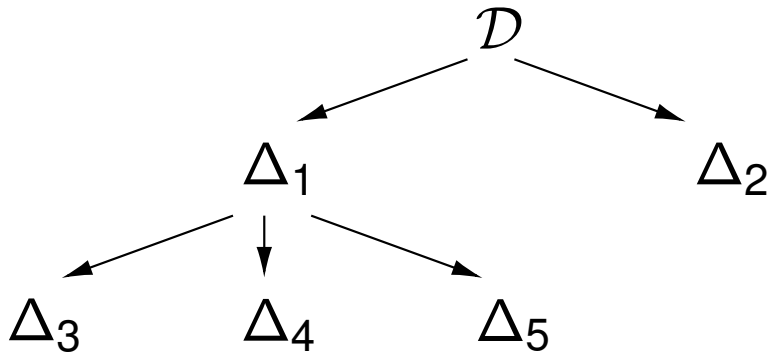
Adding a point into Δ_1 replaces Δ_1 by Δ_3, Δ_4, and Δ_5:



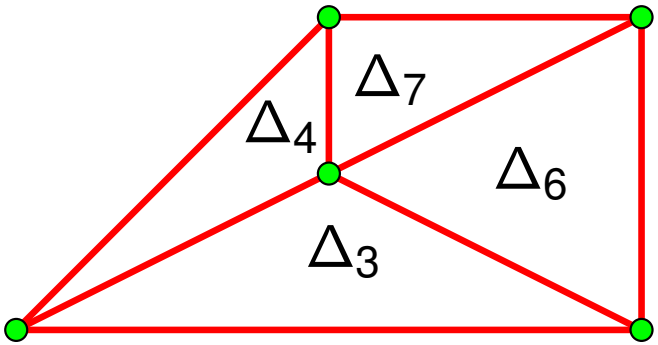


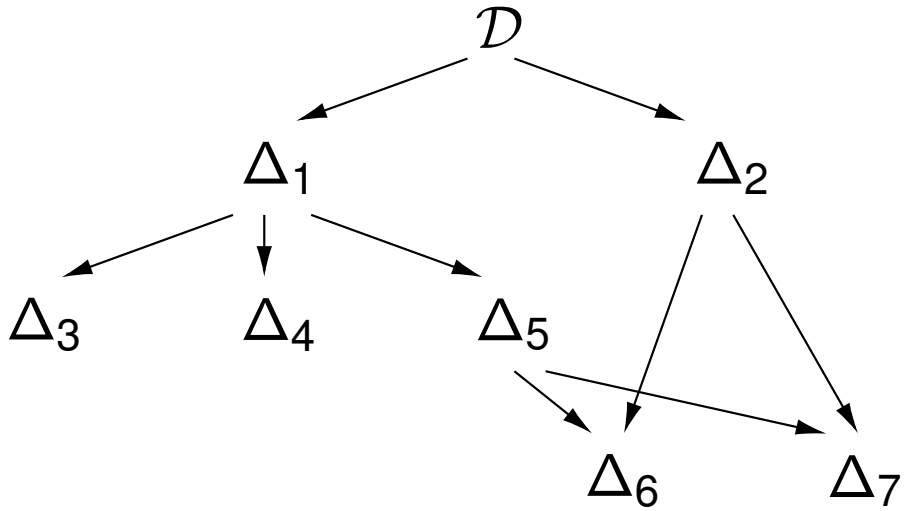
adjusting the directed acyclic graph after a flip





An edge flip replaces Δ_2 and Δ_5 by Δ_6 and Δ_7:





barycentric coordinates

Consider three points p, q, and r, not on the same line,
with coordinates $p = (p_x, p_y)$, $q = (q_x, q_y)$, and $r = (r_x, r_y)$.
A point with coordinates (x, y) lies in the interior of the triangle if and only if (x, y) can be written as a convex combination of p, q, and r.
The coefficients of the convex combination satisfy:

$$\begin{array}{ccccccccc} \begin{pmatrix} x \\ y \end{pmatrix} & = & \lambda_p \begin{pmatrix} p_x \\ p_y \end{pmatrix} & + & \lambda_q \begin{pmatrix} q_x \\ q_y \end{pmatrix} & + & \lambda_r \begin{pmatrix} r_x \\ r_y \end{pmatrix} \\ 1 & = & \lambda_p & + & \lambda_q & + & \lambda_r \end{array}$$

Exercise 3: Consider the signs of the coefficients $(\lambda_p, \lambda_q, \lambda_r)$ for points (x, y) not in the interior of the triangle (p, q, r). Start with the case of the unit triangle, $p = (0, 0)$, $q = (1, 0)$, and $r = (0, 1)$. Do the signs of λ_p, λ_q, and λ_r help in the location of (x, y) relative to the triangle?

why randomize?

The cost of locating the triangle which contains the new point is directly related to the depth of the directed acyclic graph $\mathcal{D}$.

Imagine an order of the points, so that

- adding the new point causes the deepest leaf in $\mathcal{D}$ to split, and
- then locating the triangle which contains the new point takes $O(n)$,

and so we arrive at an $O(n^2)$ algorithm.

Averaging over all $n!$ permutations of n points,
the unlucky order of points is avoided.

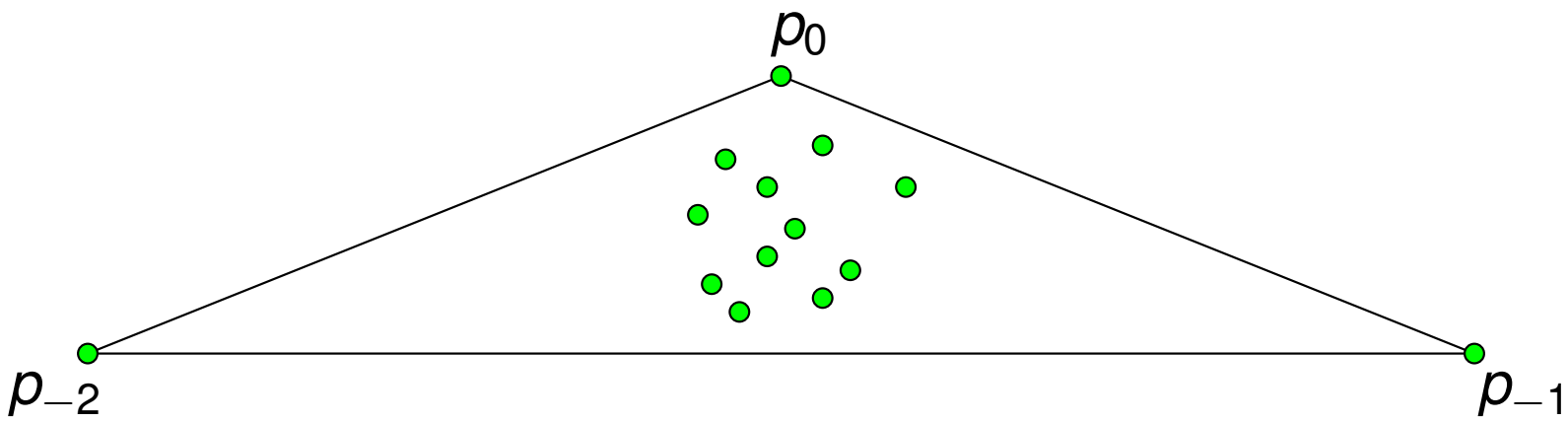
Exercise 4: For any given n, define a sequence of n points so that the k-th point in the sequence lies in the last triangle of the triangulation of the previous $k-1$ points.
Prepare such sequence as an input to the incremental algorithm of CGAL to construct a Delaunay triangulation. In runs for sufficiently large values of n, do you observe a quadratic cost?

Computing Cost of Delaunay Triangulations

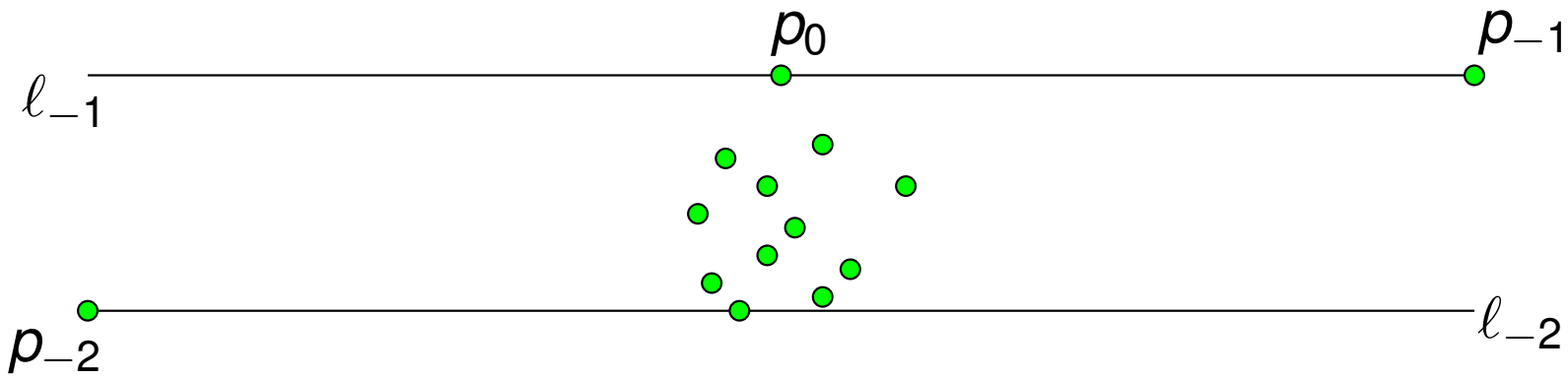
1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

the initial triangle

Instead of choosing p_0, p_{-1}, p_{-2} as below:

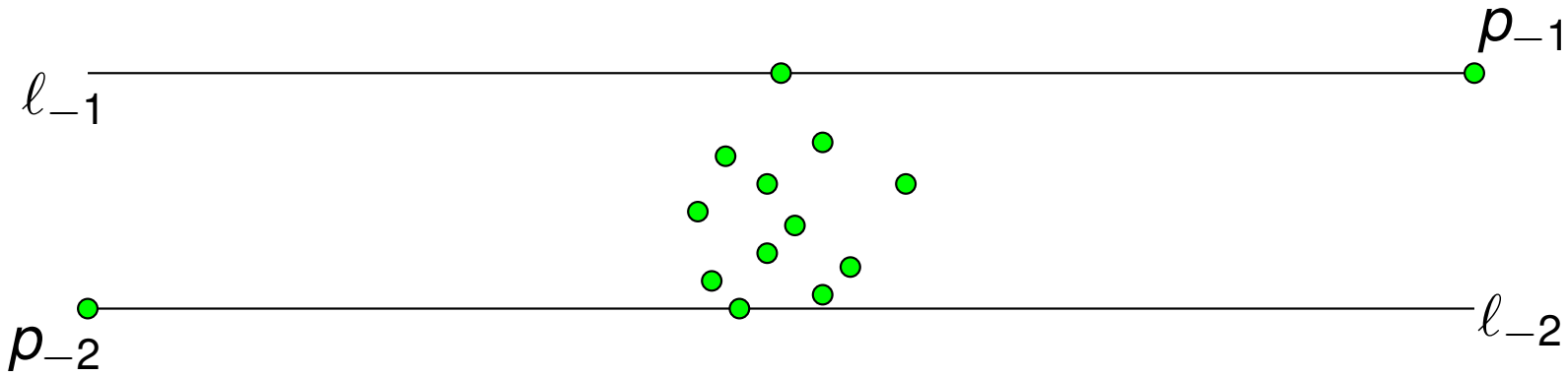


We choose p_{-1} at $(+\infty, y_{\max})$ and p_{-2} at $(-\infty, y_{\min})$



symbolic extremal points

The set P is sandwiched between two horizontal lines.



Every point on the left convex hull of P makes an edge with p_{-2}.
Every point on the right convex hull of P makes an edge with p_{-1}.

Any Delaunay triangulation of $P \cup \{p_{-1}, p_{-2}\}$ contains

- all edges connecting p_{-2} to the left convex hull of P, and
- all edges connecting p_{-1} to the right convex hull of P.

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

storage cost

Lemma (storage cost of Delaunay triangulations)

The storage cost of a Delaunay triangulation T of n points is $O(n)$.

We have two data structures:

1. The directed acyclic graph $\mathcal{D}$ occupies $O(n)$ space because every triangle is stored only once.
2. The size of T is $O(n)$ because the complexity of the Delaunay triangulation is $O(n)$.

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

the expected number of computed triangles

In the algorithm, many new triangles may arise as edges are flipped.
Even if the end result is $O(n)$,
the number of computed triangles may be much larger than $O(n)$?

Lemma (expected number of computed triangles)

For a set P of n points, the expected number of triangles computed by the algorithm DELAUNAYTRIANGULATION *is at most* $9n + 1$.

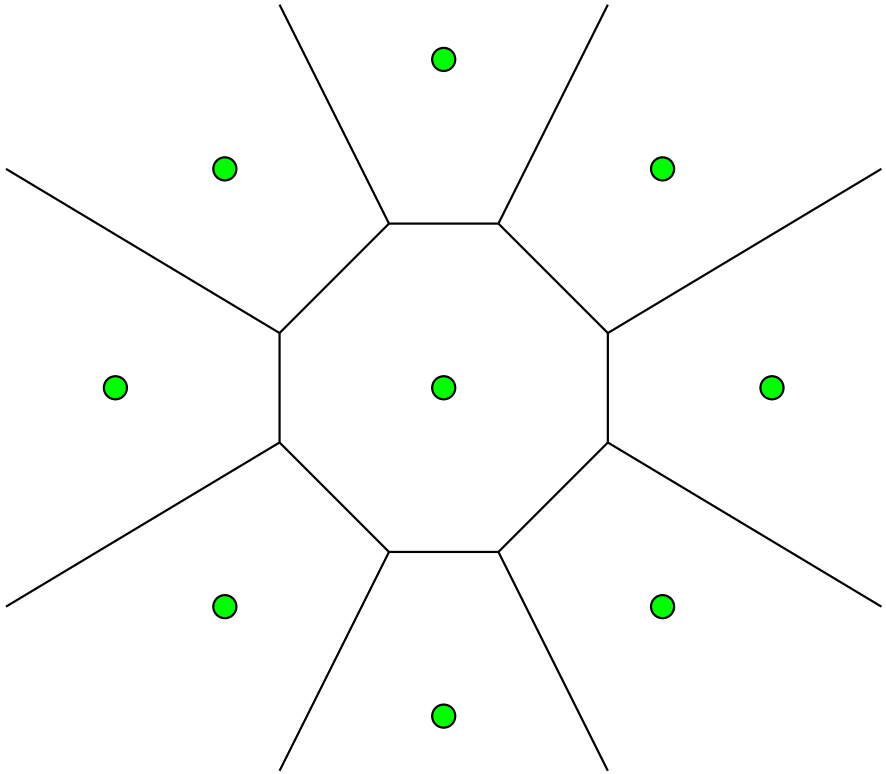
What does *expected* mean? An expected number is *an average*.

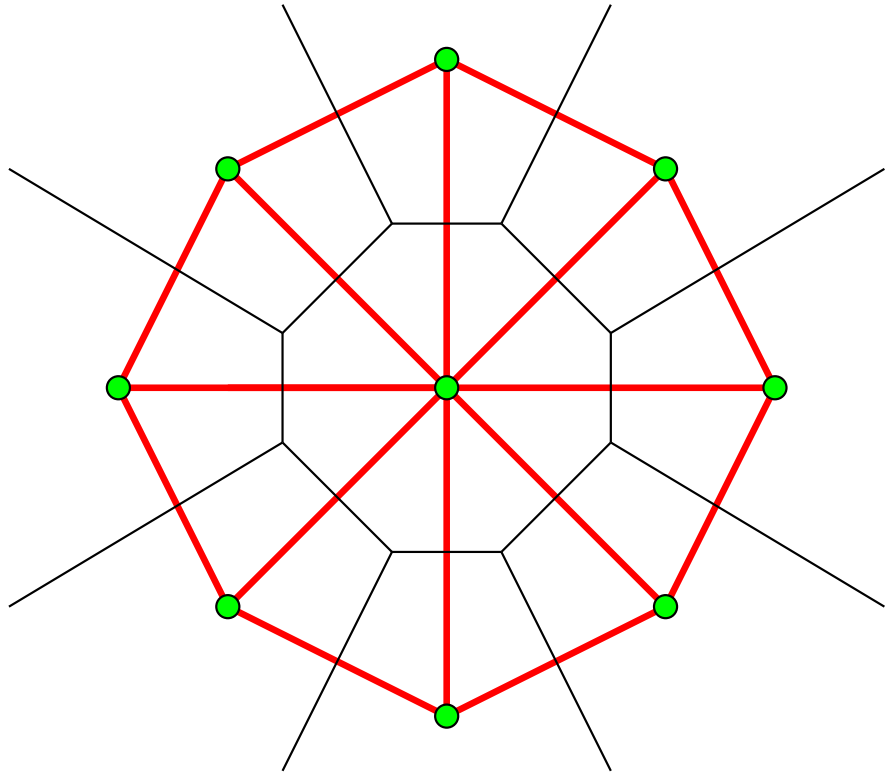
- In the main loop, r runs from 1 to n.
- What is the number of new triangles added by point p_r?
- Count the number of new triangles over the number of vertices as the average.

We have to count the average number of new triangles for r points.

the dual graph of a Voronoi diagram

At the left is a Voronoi diagram of P.





At the right is the dual graph $\mathcal{DG}(P)$ of the Voronoi diagram.

the number of new triangles and degree

Consider adding the point $p_r \in \Delta p_i p_j p_k$.

1. 3 new triangles are added as p_r is inside $\Delta p_i p_j p_k$,
2. plus the number of triangles added by flipping edges.

Every flip makes two new triangles incident to p_r.

How many new triangles?
The degree of a vertex is the number of *edges* incident to the vertex.

Let $\mathcal{DG}_r(P)$ be the current Delaunay graph,
denote $k = \deg(p_r, \mathcal{DG}_r(P))$, the degree of p_r in $\mathcal{DG}_r(P)$.

Therefore, the number of new triangles by adding p_r is $3 + 2(k - 3)$.

the expected degree — counting edges

Let $\mathcal{DG}_r(P)$ be the current Delaunay graph,
denote $k = \deg(p_r, \mathcal{DG}_r(P))$, the degree of p_r in $\mathcal{DG}_r(P)$.
What is the expected value for k? What is $E[\deg(p_r, \mathcal{DG}_r(P))]$?

Let us count edges ...

- The Delaunay graph $\mathcal{DG}_r(P)$ is the dual of the Voronoi diagram of the first r points.
- Two vertices in $\mathcal{DG}_r(P)$ are connected if and only if the Voronoi cells of their sites share an edge.

Then it follows:

$\Rightarrow$ the number of edges in $\mathcal{DG}_r(P)$ equals the number of edges of a Voronoi diagram of $r+3$ points ($+3$ because of p_0, p_{-1}, p_{-2}).

$\Rightarrow$ From the complexity of a Voronoi diagram (L-21), the number of edges is at most $3(r+3) - 6 - 3$ (-3 because of $\Delta p_0 p_{-1} p_{-2}$).

the expected degree and number of new triangles

Copying from the previous slide,
the number of edges is thus $3(r+3)-6-3 = 3r+9-9 = 3r$.

For the expected degree, we sum over all degrees of the vertices and count all edges twice, so the sum is $6r$.

$$\text{Thus: } E[\deg(p_r, \mathcal{DG}_r(P))] = 6r/r = 6.$$

We conclude:

- For $k = \deg(p_r, \mathcal{DG}_r(P))$, we derived that the number of new triangles added equals $3 + 2(k-3) = 2k-3$.
- As $E[k] = E[\deg(p_r, \mathcal{DG}_r(P))] = 6$, the expected number of new triangles is $2E[k] - 3 = 12 - 3 = 9$.

We have n points, so the total number of expected triangles is $9n+1$, the $+1$ counts the initial $\Delta p_0 p_{-1} p_{-2}$.

Q.E.D.

Computing Cost of Delaunay Triangulations

1. Delaunay Triangulations
   - definition and complexity
   - affine dependencies and edge flips
2. Computing Delaunay Triangulations
   - a randomized incremental algorithm
   - the directed acyclic graph for point location
   - the initial triangle
3. Cost Analysis
   - storage cost
   - expected number of new triangles
   - expected running time of point location

expected running time of point location

The expected number of new triangles is at most $9n + 1$.

- Making one new triangle runs in constant time.
- The total time for making all triangles is thus $O(n)$.

But what about the number of visited triangles?

The number of triangles in T at the end is linear in n,
but for the number of visited triangles, we must consider not only the triangles in T but also the number of destroyed triangles.

Denote $K(\Delta) = \{ p \in P \mid p \in \text{ circumcircle of } \Delta \}$,
as the points which will visit Δ in the point location.

The total time of point location is then $O\left(n + \sum_{\Delta} \#K(\Delta) \right)$,
where $O(n)$ is the number of triangles in T, and
the sum runs over all new triangles Δ computed by the algorithm.

expected cost of computing a Delaunay triangulation

$K(\Delta) = \{\ p \in P \mid p \in$ circumcircle of $\Delta\ \}$

Lemma (expected running time of point location)

For a set P of points in general position: $\sum_{\Delta} \#K(\Delta) = O(n \log(n))$

where the sum runs over all triangles Δ
computed by the algorithm DELAUNAYTRIANGULATION.

The above lemma, jointly with the previous lemmas implies:

Theorem (expected cost of computing a Delaunay triangulation)

A Delaunay triangulation of n points can be computed
in $O(n \log(n))$ time and $O(n)$ storage.

Let us now prove the lemma.

expected cost of point location

$K(\Delta) = \{\ p \in P \mid p \in$ circumcircle of $\Delta\ \}$

Lemma (expected running time of point location)

For a set P of points in general position: $\sum_{\Delta} \#K(\Delta) = O(n \log(n))$

where the sum runs over all triangles Δ
computed by the algorithm DELAUNAYTRIANGULATION.

$T_r = \mathcal{DG}(P_r)$, the Delaunay triangulation of the first r points in P_r.

$$\sum_{\Delta} \#K(\Delta) = \sum_{r=1}^{n} \sum_{\Delta \in T_r \setminus T_{r-1}} \#K(\Delta) = \sum_{r=1}^{n} \sum_{q \in P \setminus P_r} k(P_r, q, p_r),$$

where $k(P_r, q) = \#\{\ \Delta \in T_r \mid q \in K(\Delta)\ \}$,

$k(P_r, q, p_r) = \#\{\ \Delta \in T_r \mid q \in K(\Delta)$ and p_r incident to $\Delta\ \}$.

bounding expected values

The probability that $\Delta \in T_r$ is incident to a random point in P_r is $3/r$.

$$E[k(P_r, q, p_r)] \leq \frac{3}{r} k(P_r, q)$$

Over all $q \in P \setminus P_r$: $E\left[\sum_{\Delta \in T_r \setminus T_{r-1}} \#K(\Delta)\right] \leq \frac{3}{r} \sum_{q \in P \setminus P_r} k(P_r, q)$.

Every q is as likely to appear as p_{r+1}:

$$E[k(P_r, p_{r+1})] = \frac{1}{n-r} \sum_{q \in P \setminus P_r} k(P_r, q).$$

With $\sum_{q \in P \setminus P_r} k(P_r, q) = (n-r)E[k(P_r, p_{r+1})]$ we eliminate from the right hand side of the above inequality.

bounding the number of triangles

$$E\left[\sum_{\Delta \in T_r \setminus T_{r-1}} \#K(\Delta)\right] \leq 3\left(\frac{n-r}{r}\right) E[k(P_r, p_{r+1})]$$

$$\leq 3\left(\frac{n-r}{r}\right) E[\#(T_r \setminus T_{r+1})]$$

#destroyed triangles?

$\#T_m = 2(m+3) - 2 - 3$ (follows from complexity theorem)

$$\leq 3\left(\frac{n-r}{r}\right) E[\#(T_{r+1} \setminus T_r) - 2]$$

$\#(T_{r+1} \setminus T_r) \leq 6$

$$\leq 12\left(\frac{n-r}{r}\right),$$

and the sum for r from 1 to n gives $O(n\log(n))$, as $\sum_{r=1}^{n}(1/r) \approx \log(n)$.

Q.E.D.

summary and exercises

The cost of a randomized incremental algorithm to construct a Delaunay triangulation of n points is $O(n \log(n))$.

This is our third instance of a randomized algorithm, earlier we randomized incremental linear programming and trapezoidal maps.

We covered sections 9.3 and 9.4 in the textbook.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, 3, and 4.
2. Consult the CGAL documentation and example code on Delaunay triangulations.
3. Consider the exercises 9, 11, 16 in the textbook.

additional references

For more on triangulations, consider the beautiful book:
Triangulations. Structures for Algorithms and Applications,
by J. A. De Loera, J. Rambau, and F. Santos, Springer-Verlag, 2010.

Randomized Incremental Construction of Delaunay Triangulations of Nice Point Sets, by J.-D. Boissonat, O. Devillers, K. Dutta, and M. Glisse, *Discrete and Computational Geometry*, volume 66, pages 236–268, 2021.